

Sumário

1 Introdução

2 Modelagem dos Dados

2.1 Referencing (Referência por ObjectId)

2.2 Embedding (Documento Embutido)

3 Consultas com find()

4 Consultas Analíticas com Aggregation Pipeline

4.1 Leads por Canal de Origem

4.2 Negociações por Status

4.3 Taxa de Conversão

4.4 Leads por Atendente (com \$lookup)

4.5 Negociações por Importância

5 Considerações Finais

1

Introdução

O presente documento descreve e justifica as decisões técnicas adotadas no desenvolvimento do projeto Mini ABP, cujo objetivo é modelar, popular e consultar um sistema de gestão de leads utilizando o MongoDB como banco de dados principal.

O MongoDB é um banco de dados NoSQL orientado a documentos, amplamente utilizado em cenários que demandam flexibilidade na estrutura dos dados, escalabilidade horizontal e alto desempenho em leituras e escritas. Sua estrutura baseada em documentos JSON (BSON) permite representar dados hierárquicos de forma natural, sem a rigidez do modelo relacional.

As decisões tomadas neste projeto envolvem dois aspectos centrais: (i) a estratégia de modelagem dos dados, definindo quando embutir subdocumentos e quando referenciar outras coleções; e (ii) a escolha das formas de consulta, optando entre o método `find()` para recuperação pontual de registros e o Aggregation Pipeline para análises mais complexas.

2

Modelagem dos Dados

A modelagem em bancos de dados orientados a documentos difere substancialmente do modelo relacional. Em vez de normalizar os dados em múltiplas tabelas, o desenvolvedor deve decidir, para cada relacionamento, se os dados serão embutidos no mesmo documento (embedding) ou referenciados por identificador (referencing). Essa decisão impacta diretamente o desempenho das consultas, a consistência dos dados e a complexidade de manutenção.

2.1 Referencing (Referência por ObjectId)

A estratégia de referencing foi adotada para conectar entidades que possuem existência independente no sistema e que são reutilizadas por múltiplos documentos. As referências implementadas no projeto são:

Campo	Coleção Referenciada
leads.customerId	customers
leads.storeId	stores
leads.attendantId	users
customers.createdBy	users
negotiations.leadId	leads
logs.userId	users

Tabela 1 — Mapeamento de referências entre coleções.

A justificativa para o uso de referencing nestes casos assenta-se em três fatores principais:

- **Consistência dos dados:** entidades como users, customers e stores são atualizadas com frequência e de forma independente. Duplicar esses dados dentro de cada documento de lead ou negociação criaria inconsistências — por exemplo, a alteração do nome de um atendente exigiria atualizar todos os documentos que contêm essa informação.
- **Ciclo de vida independente:** cada entidade referenciada pode ser criada, consultada e removida sem depender dos documentos que a referenciam, o que facilita a manutenção e a governança dos dados.
- **Reutilização:** um mesmo usuário pode ser atendente de múltiplos leads e autor de múltiplos logs. Embutir esses dados geraria redundância significativa, aumentando o tamanho dos documentos sem benefício proporcional.

2.2 Embedding (Documento Embutido)

O embedding foi adotado na coleção negotiations para armazenar o histórico de estágios de cada negociação. Cada documento de negociação contém um array history com os registros de transição de status:

```
history: [ { stage: "contato_inicial", status: "aberta", changedAt: ISODate(...) },
{ stage: "proposta_enviada", status: "em_andamento", changedAt: ISODate(...) }, {
stage: "fechamento", status: "encerrada", changedAt: ISODate(...) } ]
```

A escolha pelo embedding neste contexto é fundamentada nos seguintes aspectos:

- **Pertencimento exclusivo:** o histórico de uma negociação pertence única e exclusivamente àquela negociação. Não existe cenário em que esse histórico seria acessado de forma isolada, sem o contexto do documento pai.
- **Localidade de leitura:** sempre que uma negociação é consultada, o histórico é lido junto. Embutir os dados elimina a necessidade de um join adicional com uma coleção separada, reduzindo a latência e simplificando as consultas.
- **Crescimento previsível e limitado:** o número de transições em uma negociação é finito e relativamente pequeno, não havendo risco de o documento ultrapassar o limite de 16 MB imposto pelo MongoDB.

3

Consultas com find()

O método find() foi utilizado no arquivo consultas.js para realizar buscas diretas e pontuais nas coleções. Esse método é adequado quando o objetivo é recuperar documentos com base em critérios de filtragem simples, sem necessidade de agrupamento, cálculo ou junção de dados entre coleções.

Entre as consultas implementadas, destacam-se:

- Busca de leads por canal de origem (channel).
- Filtragem de clientes criados por um determinado usuário.
- Listagem de negociações abertas ou por status específico.
- Recuperação de logs de ações de um usuário.

A escolha do `find()` para essas operações se justifica por sua eficiência em leituras diretas. Quando os campos consultados possuem índices definidos nas coleções, o MongoDB realiza a busca sem percorrer todos os documentos (full collection scan), resultando em tempo de resposta significativamente menor. Além disso, o `find()` é mais simples de escrever, ler e manter do que um pipeline de agregação, sendo a escolha natural para consultas sem necessidade de transformação ou agrupamento dos dados.

4

Consultas Analíticas com Aggregation Pipeline

O Aggregation Pipeline do MongoDB permite processar documentos em uma sequência de etapas (stages), onde a saída de cada etapa serve como entrada para a próxima. Essa abordagem é ideal para consultas analíticas que envolvem agrupamento, cálculo de métricas, junção de coleções e projeção de campos. O arquivo `aggregations.ts` implementa cinco agregações, cada uma respondendo a uma pergunta de negócio específica.

4.1 Leads por Canal de Origem

Esta agregação contabiliza quantos leads foram originados em cada canal de captação (WhatsApp, site, indicação, entre outros), ordenando os resultados do canal com maior volume para o menor.

```
db.leads.aggregate([ { $group: { _id: "$channel", total: { $sum: 1 } } }, { $sort: { total: -1 } }, { $project: { origem: "$_id", total: 1, _id: 0 } } ])
```

Justificativa: o `find()` não possui capacidade de agrupar e contar documentos. O operador `$group` com `$sum` é a única forma nativa de realizar essa contagem por categoria. O `$sort` ordena o resultado de forma descendente, facilitando a leitura, e o `$project` renomeia `_id` para `origem`, tornando o resultado mais legível.

4.2 Negociações por Status

Esta agregação distribui as negociações por status (aberta, em andamento, encerrada), permitindo uma visão geral do funil de vendas.

```
db.negotiations.aggregate([ { $group: { _id: "$status", total: { $sum: 1 } } }, { $sort: { total: -1 } }, { $project: { status: "$_id", total: 1, _id: 0 } } ])
```

Justificativa: o status da negociação reside na coleção `negotiations`, e não em `leads`. Consultar a coleção correta evita lookups desnecessários. Assim como na agregação anterior, `$group` e `$sum` são obrigatórios para consolidar a contagem por categoria.

4.3 Taxa de Conversão

Esta agregação calcula a porcentagem de negociações encerradas (convertidas) em relação ao total, fornecendo a principal métrica de desempenho do sistema de leads.

```
db.negotiations.aggregate([ { $group: { _id: null, total: { $sum: 1 }, encerradas: { $sum: { $cond: [{ $eq: ["$status", "encerrada"] }, 1, 0] } } } }, { $project: { total: 1, encerradas: 1, taxaConversao: { $multiply: [{ $divide:
```

```
["$encerradas", "$total"] }, 100] } } } ])
```

Justificativa: o operador `$cond` dentro do `$group` permite contar condicionalmente, sem necessidade de um segundo `$match`. Isso reduz o número de passagens pelo pipeline e melhora a eficiência. O cálculo da taxa acontece no `$project`, aproveitando os dois valores já acumulados — total e encerradas — em uma única passagem pelo conjunto de documentos.

4.4 Leads por Atendente (com `$lookup`)

Esta agregação conta quantos leads cada atendente possui e exibe o nome do atendente no resultado, em vez de apenas o seu `ObjectId`.

```
db.leads.aggregate([ { $group: { _id: "$attendantId", total: { $sum: 1 } } }, {  
  $lookup: { from: "users", localField: "_id", foreignField: "_id", as: "user" } }, {  
  $unwind: "$user" }, { $sort: { total: -1 } }, { $project: { atendente: "$user.name",  
    total: 1, _id: 0 } } ])
```

Justificativa: como o modelo adota referencinng, leads armazena apenas o `attendantId`. Para enriquecer o resultado com o nome do atendente, é necessário um join com a coleção `users`. O `$lookup` realiza essa junção dentro do pipeline, sem necessidade de uma segunda consulta na aplicação. O `$unwind` desaninha o array resultante do lookup, permitindo que o `$project` acesse o campo `user.name` diretamente. Esta é a abordagem recomendada pelo MongoDB para joins pontuais entre coleções.

4.5 Negociações por Importância

Esta agregação distribui as negociações pelo campo `importance` (alta, média, baixa), permitindo identificar a proporção de leads prioritários no sistema.

```
db.negotiations.aggregate([ { $group: { _id: "$importance", total: { $sum: 1 } } },  
  { $sort: { total: -1 } }, { $project: { importancia: "$_id", total: 1, _id: 0 } }  
  ])
```

Justificativa: mesmo sendo estruturalmente simples, esta consulta requer `$group` e `$sort`, operações que o `find()` não suporta. O Aggregation Pipeline é, portanto, a única forma nativa de responder a essa pergunta de negócio sem recorrer a processamento adicional na camada de aplicação.

5

Considerações Finais

As decisões de modelagem e consulta adotadas neste projeto refletem as boas práticas recomendadas para o desenvolvimento de aplicações com MongoDB. A separação entre embedding e referencinng foi guiada pelo padrão de acesso aos dados: documentos consultados sempre juntos foram embutidos; entidades com existência e ciclo de vida independentes foram referenciadas.

A escolha entre `find()` e o Aggregation Pipeline seguiu o princípio da adequação: o `find()` foi utilizado para recuperações pontuais e diretas, enquanto o Pipeline foi reservado para consultas que exigem transformação, agrupamento ou junção de dados — tarefas para as quais o `find()` não oferece

suporte nativo.

Em conjunto, essas escolhas resultam em um sistema com consultas eficientes, dados consistentes e uma estrutura de coleções que reflete adequadamente as necessidades do domínio de gestão de leads.

Projeto desenvolvido para a disciplina de Banco de Dados Não Relacional — 2025